

RabbitMQ 从入门到深入

一、为什么需要消息队列

三大核心作用

1. 异步 (Async)

同步调用：A 调 B → 等 B 执行完 → A 继续。B 耗时 500ms，A 就等 500ms。

异步调用：A 把消息放进 MQ 就返回，B 从 MQ 取消息自己处理。A 响应时间只有"发消息"的耗时 (< 1ms)。

同步 (串行)：A —[500ms]→ B —[500ms]→ C 总耗时 1000ms

异步 (MQ)：A → MQ → 立即返回 (< 1ms)

B 异步消费，C 异步消费，各自处理

2. 解耦 (Decoupling)

直接调用时，A 需要知道 B 的地址/接口，B 挂了 A 报错，B 升级 A 要改代码。

通过 MQ：A 只需往 MQ 发消息，不关心谁消费、消费者在不在线。B 挂了消息积压在 MQ，B 恢复后继续消费。

紧耦合：A —直接调用→ B (B 挂了 A 报错)

↓

松耦合：A → MQ → B (B 挂了消息堆积，恢复后继续)

3. 削峰 (Peak Shaving)

瞬时大流量 (如秒杀) 直接打 DB 会把 DB 打挂。通过 MQ 当缓冲，把流量"摊平"到时间轴上：

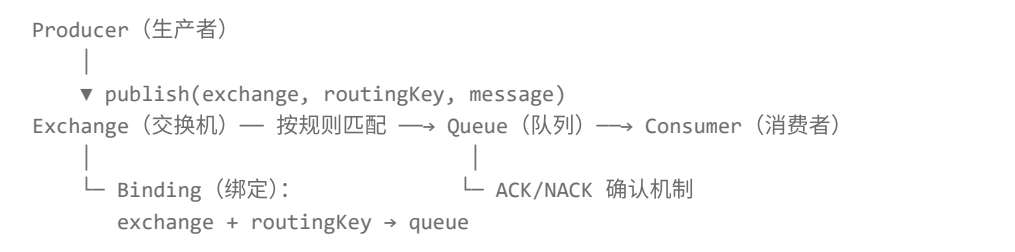
前端瞬时 10000 req/s → MQ → 后端按 DB 能处理的速率 1000 req/s 消费

什么时候不适合用 MQ

- 需要同步返回结果的场景 (查询操作)
- 强一致性要求 (ACID 事务场景，MQ 只保证最终一致)
- 消息量极小 (引入 MQ 的运维成本大于收益)

二、RabbitMQ 核心模型 (AMQP 协议)

核心组件关系



各组件职责

组件	职责
Producer	发消息，指定 exchange 和 routingKey
Exchange	接收消息，按路由规则分发到一个或多个 Queue
Queue	存储消息，消费者从这里取
Binding	Exchange 和 Queue 的绑定关系，附带一个 bindingKey
RoutingKey	生产者发消息时指定，Exchange 用它和 bindingKey 匹配决定路由
Virtual Host	相当于命名空间，不同 vhost 里的 exchange/queue 完全隔离

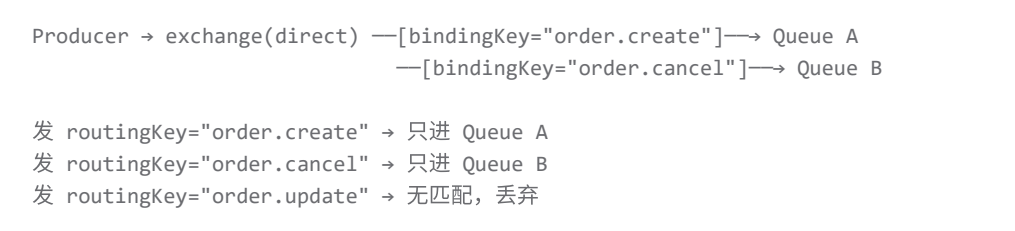
关键概念：Exchange 不存储消息

Exchange 只做路由决策，不存消息。消息必须被路由到 Queue 才能保存。如果没有 Queue 与 Exchange 绑定，或路由不匹配，消息直接丢弃（可配置 alternate-exchange 兜底）。

三、Exchange 四种类型

1. Direct Exchange（直连）

规则：routingKey 必须与 bindingKey 完全相等才路由。



适用：精确匹配，任务分发。

2. Topic Exchange（主题）

规则：routingKey 按 . 分词，bindingKey 支持通配符：

- * 匹配一个词
- # 匹配零个或多个词

```
Producer → exchange(topic)
bindingKey="order.*" → Queue A    // 匹配 "order.create"、"order.cancel"，不匹配 "order.pay.alipay"
bindingKey="order.#" → Queue B    // 匹配以 "order." 开头的一切
bindingKey="notify.#" → Queue C   // 匹配以 "notify." 开头的一切
```

项目用法：schedule.exchange 是 Topic 类型，bindingKey notify.# 匹配所有 notify.xxx 路由键：

```
notify.booking_confirmed → notify.queue
notify.booking_cancelled → notify.queue
notify.rush_created       → notify.queue
（全部被 notify.# 捕获）
```

适用：按业务域路由，多种消息类型共享一个 exchange。

3. Fanout Exchange（广播）

规则：忽略 routingKey，把消息广播到所有绑定的 Queue。

```
Producer → exchange(fanout)
→ Queue A（所有绑定的 Queue 都收到）
→ Queue B
→ Queue C
```

适用：日志广播、通知所有实例刷新缓存。

4. Headers Exchange（头部）

规则：根据消息的 headers（键值对）路由，忽略 routingKey，bindingKey 改为 arguments 匹配规则。用得极少。

四、消息可靠性：三个环节的保证

消息从 Producer 到 Consumer 经过三个环节，每个环节都可能丢消息：

```
Producer —①— Exchange —②— Queue —③— Consumer
  发送丢失       MQ宕机丢失       消费失败丢失
```

环节一：生产者 → MQ（Publisher Confirm 机制）

默认情况下 rabbitTemplate.convertAndSend() 是"发射后不管"的，MQ 收没收到不知道。

Publisher Confirms（发布确认）：MQ 收到并处理消息后，异步回调通知 Producer。

Spring AMQP 配置：

```
spring:
  rabbitmq:
    publisher-confirm-type: correlated # 每条消息单独确认（对应项目 mhp-booking 的配置）
    publisher-returns: true           # 消息无法路由到 Queue 时回调 ReturnCallback
```

回调示例：

```
rabbitTemplate.setConfirmCallback((correlationData, ack, cause) -> {
    if (!ack) {
        log.error("消息未被 MQ 接收, cause={}", cause);
        // 重发或报警
    }
});

rabbitTemplate.setReturnsCallback(returned -> {
    log.error("消息无法路由到 Queue, routingKey={}", returned.getRoutingKey());
    // 记录日志或重投
});
```

项目中： mhp-booking 的 application.yaml 配置了 publisher-confirm-type: correlated，这是生产端可靠性的基础配置。

环节二：MQ 本身（持久化）

MQ 收到消息后如果宕机，内存里的消息会丢失。必须持久化到磁盘。

三层持久化缺一不可：

```
// 1. Exchange 持久化 (durable=true)
ExchangeBuilder.topicExchange(EXCHANGE).durable(true).build();

// 2. Queue 持久化 (durable=true)
QueueBuilder.durable(NOTIFY_QUEUE).build();

// 3. 消息本身持久化 (deliveryMode=PERSISTENT)
// Spring AMQP 默认 MessageDeliveryMode.PERSISTENT，无需手动设置
```

项目中： RabbitConfig.java 里 exchange 和 queue 都设了 durable(true)，三层持久化均已配置。

环节三：消费者（手动 ACK）

消费者取到消息就开始处理，处理到一半崩溃了怎么办？

- **自动 ACK（默认）：**消息出队瞬间自动确认，Consumer 崩溃 → 消息永久丢失
- **手动 ACK：**处理完后主动调 basicAck，崩溃时消息重新入队等待重投

```
// mhp-social application.yaml
spring.rabbitmq.listener.simple.acknowledge-mode: manual

// NotifyConsumer 中手动控制
channel.basicAck(tag, false);    // 处理成功，确认
channel.basicNack(tag, false, false); // 处理失败，不重入队（进死信队列）
```

basicNack 的第三个参数 requeue：

- true：消息重新入队，会再次被消费（适合网络抖动等临时错误）
- false：消息不重入队，如果配了死信交换机则进死信队列（适合业务逻辑错误）

项目选 requeue=false 的原因：WebSocket 推送失败通常是 Consumer 端的持久性问题（用户根本不在线），无限重入队没意义，进死信队列留存供运维分析。

五、死信队列（Dead Letter Queue）

什么情况下消息变成死信

1. 消息被 Consumer 拒绝（basicNack / basicReject + requeue=false）
2. 消息在 Queue 里超过了 TTL（time-to-live）
3. Queue 达到最大长度，新消息被挤出

死信队列配置（项目实际代码）

正常流程：

```
schedule.exchange (Topic)
    "notify.#" → notify.queue (x-dead-letter-exchange=schedule.dead.exchange)
```

死信流程：

```
notify.queue 里的消息 basicNack(requeue=false)
    → 自动路由到 schedule.dead.exchange (Direct)
        "notify.dead.queue" → notify.dead.queue
```

// 死信交换机：Direct 类型

```
DirectExchange scheduleDeadExchange = ExchangeBuilder.directExchange("schedule.dead.exchange").durable(true).bu
```

// 正常队列：声明死信交换机参数

```
Queue notifyQueue = QueueBuilder.durable("notify.queue")
    .withArgument("x-dead-letter-exchange", "schedule.dead.exchange")
    .withArgument("x-dead-letter-routing-key", "notify.dead.queue")
    .build();
```

// 死信队列：普通持久化队列

```
Queue notifyDeadQueue = QueueBuilder.durable("notify.dead.queue").build();
```

死信队列的价值

- **消息不丢失**：处理失败的消息不会消失，而是沉淀到死信队列
- **问题可追溯**：运维可以查看死信队列里的消息，分析失败原因
- **人工重投**：分析完问题后，可以把死信队列里的消息手动重新投递

六、延迟消息（TTL + DLX 模式）

RabbitMQ 原生不支持延迟队列，常用两种方案实现：

方案一：TTL + 死信队列（项目未用，但面试常问）

```
Producer → delay.queue (TTL=30min, DLX=process.exchange)
      |
      | 消息超过 TTL，变成死信
      |
process.exchange → process.queue → Consumer (30分钟后处理)
```

缺点：所有消息必须用同一个 TTL（Queue 级别设置），不能每条消息单独设不同延迟。如果每条消息设不同 TTL（Message 级别），由于 RabbitMQ 按队列头部过期，中间有短TTL消息也无法提前过期。

方案二：rabbitmq-delayed-message-exchange 插件

安装插件后，Exchange 支持 `x-delayed-message` 类型，每条消息可设不同延迟时间：

```
rabbitTemplate.convertAndSend(EXCHANGE, routingKey, msg, message -> {
    message.getMessageProperties().setDelay(1800000); // 30分钟，毫秒
    return message;
});
```

项目中的定时任务选型：本项目用的是 XXL-Job（预约超时取消、每日提醒），而非延迟消息。XXL-Job 擅长周期性批量任务；延迟消息擅长"N 分钟后针对某条数据触发"的精细化延迟。

七、消费者幂等性

为什么会重复消费

RabbitMQ 保证 **At-Least-Once**（至少一次）投递语义：

- Consumer 处理消息后 crash，来不及 ACK → MQ 重投
- 网络抖动，MQ 没收到 ACK → MQ 重投
- Consumer 手动 NACK + `requeue=true` → 重投

同一条消息可能被消费多次，Consumer 必须保证幂等。

幂等性设计方案

方案一：Redis SET NX 去重（项目采用）

```
String idempotentKey = "msg:processed:" + msg.getMsgId();

// setIfAbsent = SET NX (Not eXists), 原子操作
Boolean isNew = stringRedisTemplate.opsForValue()
    .setIfAbsent(idempotentKey, "1", 24, TimeUnit.HOURS);

if (Boolean.FALSE.equals(isNew)) {
    // key 已存在 → 这条消息已处理过 → 直接 ACK 跳过
    channel.basicAck(tag, false);
    return;
}
// key 不存在 → 首次处理 → 继续业务逻辑
```

msgId 在 MQSender 里用 IdUtil.fastSimpleUUID() 生成，每次发消息时设置，MQ 重投时 msgId 不变。

注意：项目在处理失败时删除幂等 key (stringRedisTemplate.delete(idempotentKey))，允许下次重投时再处理，避免因一次瞬时故障就永久跳过这条消息。

方案二：数据库唯一索引

消息体里携带业务唯一键，插入 DB 时依赖唯一索引报 DuplicateKeyException 来防重。

方案三：状态机检查

处理前查状态，只有特定状态才处理（适合有状态的业务流程）。

八、消息积压处理

什么情况下会积压

Consumer 消费速度 < Producer 发送速度，消息在 Queue 里堆积。常见原因：

- Consumer 挂了（积压最严重）
- 业务逻辑处理变慢（DB 慢查询、下游超时）
- 流量突增

处理方案

紧急处理（救火）：

1. 扩容 Consumer 实例数（最快）
2. 如果 Consumer 挂了，先修复重启

3. 消息已过期或业务允许丢弃：清空队列，接受损失

预防积压（治本）：

1. 设置 Queue 最大长度（`x-max-length`），防止无限积压
2. Consumer 限流：Spring AMQP 的 `prefetch-count`（每次只预取 N 条）

```
spring.rabbitmq.listener.simple.prefetch: 1 # Consumer 每次只取 1 条，处理完再取下一条
```

九、Spring AMQP 编码实践

配置声明

```
@Configuration
public class RabbitConfig {
    // 1. 声明 Exchange
    @Bean
    public TopicExchange myExchange() {
        return ExchangeBuilder.topicExchange("my.exchange").durable(true).build();
    }

    // 2. 声明 Queue（带死信配置）
    @Bean
    public Queue myQueue() {
        return QueueBuilder.durable("my.queue")
            .withArgument("x-dead-letter-exchange", "my.dead.exchange")
            .withArgument("x-dead-letter-routing-key", "my.dead.key")
            .build();
    }

    // 3. 绑定
    @Bean
    public Binding myBinding(Queue myQueue, TopicExchange myExchange) {
        return BindingBuilder.bind(myQueue).to(myExchange).with("my.#");
    }
}
```

生产者发消息

```
@Component
@RequiredArgsConstructor
public class MyProducer {
    private final RabbitTemplate rabbitTemplate;

    public void send(String data) {
        MyMessage msg = new MyMessage();
        msg.setMsgId(UUID.randomUUID().toString()); // 幂等 ID
        msg.setData(data);
        rabbitTemplate.convertAndSend("my.exchange", "my.order.create", msg);
    }
}
```



```
}
```

消费者（手动 ACK）

```
@Component
@RequiredArgsConstructor
public class MyConsumer {
    private final StringRedisTemplate redisTemplate;

    @RabbitListener(queues = "my.queue")
    public void onMessage(MyMessage msg, Message rawMsg, Channel channel) throws IOException {
        long tag = rawMsg.getMessageProperties().getDeliveryTag();
        String key = "msg:processed:" + msg.getMsgId();

        // 幂等检查
        Boolean isNew = redisTemplate.opsForValue().setIfAbsent(key, "1", 24, TimeUnit.HOURS);
        if (Boolean.FALSE.equals(isNew)) {
            channel.basicAck(tag, false); // 重复，直接 ack 跳过
            return;
        }

        try {
            // 业务逻辑
            doProcess(msg);
            channel.basicAck(tag, false); // 成功，确认
        } catch (Exception e) {
            redisTemplate.delete(key); // 删除幂等 key，允许下次重试
            log.error("消息处理失败", e);
            channel.basicNack(tag, false, false); // 失败，进死信队列
        }
    }
}
```

basicAck / basicNack 参数说明

```
channel.basicAck(deliveryTag, multiple);
// deliveryTag: 消息唯一序号（从 rawMsg 里取）
// multiple: true=确认所有 <= deliveryTag 的消息；false=只确认这一条

channel.basicNack(deliveryTag, multiple, requeue);
// requeue: true=重新入队；false=进死信队列（如果配了 DLX）

channel.basicReject(deliveryTag, requeue);
// 和 basicNack 类似，但只能拒绝单条，没有 multiple 参数
```

十、项目实战：预约通知完整链路

完整流程图



六种消息类型

类型	触发方	目标	说明
BOOKING_CONFIRMED	商家确认预约	客人	"您的预约已被商家确认"
BOOKING_COMPLETED	商家标记完成	客人	"预约已完成，欢迎评价"
BOOKING_CANCELLED	任意一方取消	另一方	商家取消通知客人，客人取消通知商家
SCHEDULE_REMINDER	XXL-Job 每日 8:30	明日有档期的客人	"明日档期提醒"
RUSH_CREATED	商家创建抢档期	所有关注者（fan-out）	消费端查 follow 表遍历推送
RUSH_REMINDER	XXL-Job	所有关注者（fan-out）	"5 分钟后抢档期开放"

fan-out 为什么在消费端做，不在生产端发多条

生产端 fan-out：MQSender 查出所有 follower，发 N 条消息（N = 粉丝数）。

消费端 fan-out（项目采用）：MQSender 发 1 条消息（只带 merchantId），NotifyConsumer 收到后查 follow 表，逐一推 WebSocket。

选消费端的原因：

1. **解耦**：booking 服务不需要查 follow 表（follow 数据在 social 服务），避免跨服务查询
2. **发送可靠**：只需确认 1 条消息被 MQ 接收，不是 N 条
3. **灵活**：消费端推送策略（在线/离线）只需改 social 服务，不影响 booking

十一、常见面试题

Q：RabbitMQ 如何保证消息不丢失？

三个环节三种保证：① 生产者 Publisher Confirm + Return Callback ② MQ 自身 exchange/queue/message 持久化 ③ 消费者手动 ACK（处理完再确认）。

Q：如何保证消费者幂等？

消息携带唯一 msgId，Consumer 处理前用 Redis SET NX 写入 msg:processed:{msgId}，返回 false 说明已处理，直接 ACK 跳过。

Q：死信队列有什么用？

消息在 basicNack(requeue=false) / TTL 过期 / 队列满时，自动路由到死信队列，消息不丢失。运维可以查死信队列分析失败原因，必要时手动重投。

Q：Topic Exchange 和 Direct Exchange 有什么区别？

Direct 精确匹配 routingKey；Topic 支持通配符（* 一个词，# 零或多个词），一个 Exchange 可路由多种消息到不同 Queue。

Q：手动 ACK 和自动 ACK 的区别，什么时候用手动？

自动 ACK：消息出队即确认，Consumer 挂了消息丢失。手动 ACK：处理完后才确认，失败可重投或进死信。只要业务不允许消息丢失就用手动 ACK。

Q：什么是消息积压？如何处理？

Consumer 消费速度跟不上 Producer 发送速度导致 Queue 堆积。紧急：扩容 Consumer 实例。预防：Consumer 加 prefetch=1 限流、设 Queue 最大长度。

Q：publisher-confirm-type: correlated 和 simple 的区别？

simple：所有消息共享一个确认回调，无法区分哪条消息失败。correlated：每条消息有独立的 CorrelationData，可以精确追踪哪条消息未确认。生产环境应用 correlated。